

Introduction: What is Recursion?

Recursion is a technique where a function **calls itself** to solve a smaller version of the same problem. It is one of the most powerful ideas in programming and is essential for working with trees, graphs, and divide-and-conquer algorithms.

This lab covers:

1. **Anatomy of a recursive function** — base case and recursive case.
2. **The call stack** — how function calls build up and unwind.
3. **Classic examples** — factorial, Fibonacci, power.
4. **Recursion on arrays and strings** — sum, search, reverse.

Part 1: Structure of a Recursive Function

1.1 Base Case and Recursive Case

Two Parts Every Recursive Function Must Have

- **Base case** — the simplest input, answered directly without another call. Without this the function recurses forever and causes a *stack overflow*.
- **Recursive case** — reduces the problem to a smaller version and calls itself. Every call must move *closer* to the base case.

Part	What it does
Base case	Stops the recursion; returns a direct answer
Recursive case	Calls the same function with a <i>smaller</i> argument

1.2 Example 1 — Factorial

Listing 1: Factorial: the simplest recursive function

```
1 #include <iostream>
2 using namespace std;
3
4 // n! = n * (n-1) * ... * 1, with 0! = 1
5 int factorial(int n) {
6     if (n == 0) return 1;           // base case
7     return n * factorial(n - 1);    // recursive case
8 }
9
10 int main() {
11     for (int i = 0; i <= 7; i++)
12         cout << i << "! = " << factorial(i) << "\n";
13 }
```

Expected output:

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
```

Call-stack trace for factorial(4)		
Step	Call	Returns
1	factorial(4) calls factorial(3)	waiting...
2	factorial(3) calls factorial(2)	waiting...
3	factorial(2) calls factorial(1)	waiting...
4	factorial(1) calls factorial(0)	waiting...
5	factorial(0) — base case hit	1
6	factorial(1) returns 1 × 1	1
7	factorial(2) returns 2 × 1	2
8	factorial(3) returns 3 × 2	6
9	factorial(4) returns 4 × 6	24

Base case Descending (calls pending) Ascending (returning values)

Part 2: Classic Recursive Problems

2.1 Fibonacci Numbers

Listing 2: Fibonacci: two base cases and two recursive calls

```
1 #include <iostream>
2 using namespace std;
3
4 // F(0)=0, F(1)=1, F(n) = F(n-1) + F(n-2)
5 int fib(int n) {
6     if (n == 0) return 0;           // base case 1
7     if (n == 1) return 1;           // base case 2
8     return fib(n - 1) + fib(n - 2); // recursive case
9 }
10
11 int main() {
12     cout << "Fibonacci sequence:\n";
13     for (int i = 0; i <= 10; i++)
14         cout << "F(" << i << ") = " << fib(i) << "\n";
15 }
```

Expected output:

Fibonacci sequence:

F(0) = 0
F(1) = 1
F(2) = 1
F(3) = 2
F(4) = 3
F(5) = 5
F(6) = 8
F(7) = 13
F(8) = 21
F(9) = 34
F(10) = 55

2.2 Power Function

Listing 3: Compute base^{exp} recursively

```
1 #include <iostream>
2 using namespace std;
3
4 // base^exp: multiply base by itself exp times
5 int power(int base, int exp) {
6     if (exp == 0) return 1;           // base case: anything^0 = 1
7     return base * power(base, exp - 1); // recursive case
8 }
9
```

```
10 int main() {  
11     cout << "2^10 = " << power(2, 10) << "\n";    // 1024  
12     cout << "3^4  = " << power(3, 4)  << "\n";    // 81  
13     cout << "5^0  = " << power(5, 0)  << "\n";    // 1  
14 }
```

Expected output:

2¹⁰ = 1024

3⁴ = 81

5⁰ = 1

Part 3: Recursion on Arrays

3.1 Sum and Search

Array Recursion Pattern

To process an array of size n recursively, treat it as:

- **Base case:** empty array (index = size) — return identity value.
- **Recursive case:** process the first element, then recurse on the rest.

Listing 4: Recursive sum and linear search — no loops allowed

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Sum of arr[i], arr[i+1], ..., arr[n-1]
6 int sum(const vector<int>& arr, int i = 0) {
7     if (i == (int)arr.size()) return 0;    // base case: nothing left
8     return arr[i] + sum(arr, i + 1);      // head + sum of rest
9 }
10
11 // Find index of target, or -1 if not found
12 int search(const vector<int>& arr, int target, int i = 0) {
13     if (i == (int)arr.size()) return -1;  // base case: not found
14     if (arr[i] == target) return i;       // base case: found
15     return search(arr, target, i + 1);    // search the rest
16 }
17
18 int main() {
19     vector<int> v = {4, 2, 7, 1, 9, 3};
20     cout << "Array: 4 2 7 1 9 3\n";
21     cout << "Sum:          " << sum(v)          << "\n";    // 26
22     cout << "Index of 7:   " << search(v, 7)      << "\n";    // 2
23     cout << "Index of 5:   " << search(v, 5)      << "\n";    // -1
24 }

```

Expected output:

```

Array: 4 2 7 1 9 3
Sum:          26
Index of 7:   2
Index of 5:   -1

```

Part 4: Recursion on Strings

4.1 Reverse and Palindrome

Listing 5: Reverse a string and check palindrome recursively

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Reverse s by reversing the tail and appending the head
6 string reverse(const string& s) {
7     if (s.size() <= 1) return s;           // base case
8     return reverse(s.substr(1)) + s[0];    // tail reversed + head
9 }
10
11 // Check palindrome by comparing outer characters and moving inward
12 bool isPalindrome(const string& s, int lo, int hi) {
13     if (lo >= hi) return true;             // base case: 0 or 1 chars left
14     if (s[lo] != s[hi]) return false;      // mismatch found
15     return isPalindrome(s, lo + 1, hi - 1); // check inner part
16 }
17
18 int main() {
19     cout << reverse("hello") << "\n";    // olleh
20     cout << reverse("recursion") << "\n"; // noisrucer
21
22     string w1 = "racecar", w2 = "hello";
23     cout << w1 << " is palindrome? "
24         << (isPalindrome(w1, 0, w1.size()-1) ? "yes" : "no") << "\n";
25     cout << w2 << " is palindrome? "
26         << (isPalindrome(w2, 0, w2.size()-1) ? "yes" : "no") << "\n";
27 }
```

Expected output:

olleh

noisrucer

racecar is palindrome? yes

hello is palindrome? no

Common Mistakes in Recursion

Mistake	What goes wrong
Missing base case	Function recurses forever — stack overflow crash
Wrong base case	Returns wrong answer for smallest inputs
Not shrinking	Argument stays the same — infinite loop
Off-by-one	Base case triggers too early or too late

Lab Exercises

Exercise 1

[**Factorial — Call Stack Trace**] Without running any code, manually trace the call stack for `factorial(5)`.

- Draw each stack frame as it is pushed onto the call stack, showing the local variable `n` and the expression waiting to be completed.
- Then show each frame being popped as values are returned.

After completing the trace, implement `factorial` and add a `cout` statement at the start of each call that prints:

```
Entering factorial(n)
```

and another at the end that prints:

```
Returning factorial(n) = result
```

Run the program for `factorial(5)` and confirm your manual trace matches the output.

Exercise 2

[**Fibonacci — Count Calls**] Implement `fib(n)` exactly as shown in Example 2. Add a global counter variable that increments every time `fib` is called.

- Print `fib(n)` and the total call count for $n \in \{1, 5, 10, 15, 20\}$.
- Fill in the table and describe the growth pattern you observe.

n	<code>fib(n)</code>	Total calls to <code>fib</code>
1		
5		
10		
15		
20		

Why does the number of calls grow so quickly? Hint: draw the first few levels of the recursion tree for `fib(5)`.

Exercise 3

[**Power Function — Extend and Verify**] Using the recursive `power(base, exp)` from Example 3:

- Verify the output for: `power(2,0)`, `power(2,5)`, `power(3,3)`, `power(10,4)`.

- ii. Trace the calls made for `power(2, 4)` step by step, showing the argument values and return values at each level.
- iii. Modify the function to also handle a **negative exponent** by computing $\frac{1}{\text{base}^{|\text{exp}|}}$ and returning a double. Verify with `power(2.0, -3)` which should give 0.125.

Exercise 4

[Recursive Array Operations] Implement the following functions **recursively** (no loops allowed):

- i. `int maxVal(const vector<int>& arr, int i = 0)` — returns the maximum element.
- ii. `bool isSorted(const vector<int>& arr, int i = 0)` — returns `true` if the array is sorted in ascending order.
- iii. `void printReverse(const vector<int>& arr, int i)` — prints all elements in reverse order without modifying the array.
- iv. `int countEvens(const vector<int>& arr, int i = 0)` — returns how many even numbers are in the array.

Test each function with the array {3, 7, 1, 9, 4, 6, 2} and print the results.

Exercise 5

[Sum of Digits] Write a recursive function:

```
int sumDigits(int n);
```

that returns the sum of all digits of a non-negative integer. For example: `sumDigits(1234) = 10` because $1 + 2 + 3 + 4 = 10$.

- Identify the base case: what is the smallest input and what should be returned directly?
- Identify the recursive case: how do you extract one digit and shrink the problem?
- Test with: `sumDigits(0)`, `sumDigits(9)`, `sumDigits(123)`, `sumDigits(9999)`.

Exercise 6

[String Recursion] Implement the following string functions **recursively**:

- i. `int countChar(const string& s, char c, int i = 0)` — returns the number of times character `c` appears in `s`. Test: `countChar("banana", 'a')` should return 3.
- ii. `string removeChar(const string& s, char c)` — returns a new string with every occurrence of `c` removed. Test: `removeChar("hello world", 'l')` should return "heo word".
- iii. `string toBinary(int n)` — converts a positive integer to its binary string representation. For example: `toBinary(10)` should return "1010". *Hint: the last bit is $n \% 2$; recurse on $n / 2$.*

Exercise 7

[Identify the Bug] Each function below is supposed to work recursively but contains exactly **one bug**. Find the bug, explain what goes wrong, and write the corrected version.

- i. **Buggy factorial:**

```
int factorial(int n) {  
    return n * factorial(n - 1);    // missing base case  
}
```

What happens when you call `factorial(3)`? Fix it.

ii. **Buggy reverse:**

```
string reverse(const string& s) {  
    if (s.size() == 0) return s;  
    return s[0] + reverse(s.substr(1));    // wrong order  
}
```

What does this return for "abc"? What should it return? Fix it.

iii. **Buggy sum:**

```
int sum(const vector<int>& arr, int i = 0) {  
    if (i == (int)arr.size()) return 0;  
    return sum(arr, i + 1);    // forgets to include arr[i]  
}
```

What does this return for {1,2,3}? What should it return? Fix it.

Exercise 8

[Print a Countdown and Countup] Write two recursive functions:

i. `void countdown(int n)` — prints:

3
2
1
Go!

for `countdown(3)`. The number is printed *before* the recursive call.

ii. `void countup(int n)` — prints:

Go!
1
2
3

for `countup(3)`. The number is printed *after* the recursive call.

iii. Run both with $n = 5$ and record the output. Then explain in one sentence why swapping the position of `cout` (before vs. after the recursive call) reverses the order of output.
